2021 final | Midterm solutions

Answers and explanations for the 22 core questions and 3 optional questions selected in our earlier question packet. Original numbering is retained. This is an independently prepared solution, not an official marking scheme.

Quick answer key

Question	Answer	Question	Answer
1	a - True	20	d
2	b - False	21	b, c, d (wording caveat)
3	b - False	22	a, c
4	b - False as written	24	a, c, d
5	b - False	25	a, b, c
6	a - True	26	a, c, d
7	a - True (ordinary built-ins)	27	c, d
9	b - False	30	a only; output 11
11	c - 7	31	2041 then 0033
12	c - private	32	Full table on page 6
14	b - 64 GB	Optional 10	b - False
15	b - 0x2024 and 2021	Optional 17	d - Green, 33
		Optional 23	b, c

How to use this solution

Attempt the questions first, then compare your reasoning. For Q31, track the pointer position and array contents separately. For Q32, write the type beside each expression before doing address arithmetic.

Important qualifications

Q4: Returning the stream by reference is correct, but the statement about copying is false for std::ostream. An old marking scheme might follow an oversimplified explanation.
Q21: b, c, d fits the word “distinguishing.” Some courses group all four choices as OOP pillars; no official key was supplied.
Q26: A pointer stores the allocated address; it does not generally choose where the allocator places an object.

True / false | Q1-Q9

Q1 | a - True

Under the introductory model used in the question, a variable names storage for a value of a specified type. Its declaration must be visible before it is used.

Q2 | b - False

A function may contain several return statements. Executing any one of them exits that invocation; there is no rule limiting the source code to one return statement.

```
int absolute(int x) {
    if (x < 0) return -x;
    return x;
}
```

Q3 | b - False: compilation error

The + operators group left to right. The first operation is "hello" + ",", involving two string literals, not a std::string. C++ does not define concatenation by adding two pointers. The later std::string("world") does not repair that earlier invalid expression.

```
std::string str2 = std::string("hello") + "," + "world";
```

Q4 | b - False as written

A normal insertion overload returns std::ostream& so that it returns the same stream and supports chaining. However, std::ostream has a deleted copy constructor. Returning an existing stream by value does not simply make a new copy: it fails to compile. The first part describes the right pattern; the last claim makes the full statement false. [1]

```
std::ostream& operator<<(std::ostream& out, const Point& pt) {
    out << pt.x << "," << pt.y;
    return out;
}
```

Q5 | b - False

Effective size is the number of elements in use; allocated size is capacity. Effective size can equal capacity when the storage is full. For a valid representation: 0 <= effective size <= allocated size.

Q6 | a - True

Each declarator needs its own *. Thus int* p1, p2 declares a pointer p1 and an int p2. Whitespace does not change the meaning: p3 is a pointer and p4 is an int.

Q7 | a - True in the intended setting

For ordinary objects and built-in operators, &x obtains x's address and * dereferences it, so *&x denotes x. Similarly, &*p recovers the pointer value. This introductory identity is not permission to read through invalid pointers; overloaded operators can also behave differently.

Q9 | b - False

begin() denotes the first element; end() denotes the position one past the last element. end() is a stopping marker and must not be dereferenced.

Short questions | Q11-Q22

Q11 | c - 7

Multiplication, division and remainder have equal precedence and associate left to right. The left division is integer division; 1.0 only makes the right-hand parenthesized expression floating point.

```
9 * (8-7) / 6        = 9 / 6 = 1
5 * 4 % 3            = 20 % 3 = 2
2 + 1.0              = 3.0
Total                = 1 + 2 * 3.0 = 7.0
```

Q12 | c - private

Private data members support encapsulation: public member functions control how the object's state can be observed or changed and can preserve its invariants. friend is not an access level.

Q14 | b - 64 GB

A 36-bit address gives 2^36 distinct byte addresses. Using the question's binary unit definition, 2^36 / 2^30 = 2^6 = 64 GB. This is the theoretical addressable capacity, not necessarily installed memory.

Q15 | b - 0x2024 and 2021

pi stores the address of i; ppi stores the address of pi. Dereferencing ppi once gives pi; twice gives i's value.

```
pi   = &i  = 0x2020
ppi  = &pi = 0x2024
*ppi = pi  = 0x2020
**ppi = i  = 2021
```

Q20 | d - Constant call by reference

Use const T&: this avoids copying the object and prevents modification through that parameter. A non-const reference allows changes; passing by value creates a separate parameter object.

```
void inspect(const LargeObject& object);
```

Q21 | b, c, d - with a wording caveat

Encapsulation, inheritance and polymorphism are the likely intended distinguishing OOP features. Abstraction is also fundamental to OOP, but it is shared with procedural and other programming styles. Because the question explicitly asks what distinguishes OOP, b, c, d is the best-fit selection. If the instructor uses the broader “four pillars” convention, a, b, c, d may be expected; the official marking is unverified.

Q22 | a, c - Number and types of parameters

For the ordinary function overloading in this question, parameter names do not distinguish overloads, and changing only the return type is not enough. Different numbers or types of parameters can distinguish them.

Memory and classes | Q24-Q30

Q24 | a, c, d

a: typename is a reserved keyword.
b: type_name is legal: underscores are allowed.
c: type-name is not a single identifier; the hyphen is a minus token.
d: 2typenames is illegal because an identifier cannot begin with a digit.

Q25 | a, b, c

The lecture memory model distinguishes static storage, heap storage for dynamic allocation, and stack storage for ordinary function activations/local variables. A queue is a data structure, not a separate memory region. These are the course's conventional implementation terms; the C++ language describes storage durations more abstractly.

Q26 | a, c, d

a: Dynamic allocation reserves memory at runtime and returns an address that a pointer can hold.
b: False in ordinary allocation: new/malloc select the storage location. A pointer records that address; it does not generally choose an arbitrary heap address for the object.
c: A pointer compactly refers to a large structure without copying it.
d: Pointers can represent connections between nodes, as in a linked list.

Reading caveat: “specify” in option b is imprecise. The answer above reads it as choosing the allocation address. Merely storing/referencing an address would be true, but is a different claim. Placement new constructs in existing storage; it does not make ordinary heap allocation choose any requested address.

Q27 | c, d - &p and (p+1)

Assume p is an ordinary raw pointer and the relevant operations are valid. p is an lvalue naming a pointer object. *p is an lvalue denoting the pointed-to object. &p and p+1 produce pointer values (prvalues); neither denotes a pointer object you can assign into. Also, not every lvalue is modifiable: a const object is still an lvalue.

```
p = anotherPointer; // potentially valid
*p = 5;             // potentially valid for a writable int
// &p = ...;        // invalid assignment target
// (p+1) = ...;     // invalid assignment target
```

Q30 | a only - exact output: 11

Construction proceeds through the bases before the derived constructor. A sets a = 1; B sets b = 2; C changes b to 1 and sets c = 3. The final object therefore has a = 1, b = 1, c = 3.

pb has static type B*. B::display is **not virtual**, so pb->display() selects B::display. Declaring C::display virtual does not retroactively make B::display virtual. B prints a and b with no separator, followed by a newline. Neither 2 nor 3 is printed.

```
11
```

Q31 | Pointer tracing step by step

Exact output:

```
2041
0033
```

Initial state: arr = {2, 0, 2, 1}; p points to arr[0]. Each declaration is a separate full expression, so its increment is complete before the next declaration begins.

Statement	Assigned value	p afterwards	arr afterwards
int a = *++p;	a = 0	&arr[1]	{2,0,2,1}
int b = *p++;	b = 0	&arr[2]	{2,0,2,1}
int c = ++*p;	c = 3	&arr[2]	{2,0,3,1}
int d = (*p)++;	d = 3	&arr[2]	{2,0,4,1}

Q31a | *++p means *(++p)

Prefix ++ advances p from arr[0] to arr[1]. Dereferencing that new pointer reads 0, so a becomes 0.

Q31b | *p++ means *(p++)

Postfix ++ supplies the old pointer value for dereferencing. That reads arr[1], which is 0, so b becomes 0. The pointer itself advances to arr[2].

Q31c | ++*p means ++(*p)

Dereference p to access arr[2]. Prefix increment changes the element from 2 to 3 and produces the new value, so c becomes 3. p does not move.

Q31d | (*p)++ changes the element

Postfix increment produces the element's old value, 3, so d becomes 3. The element then becomes 4. p still points to arr[2].

Why there are no spaces

The first cout joins the final array values 2, 0, 4, 1 as **2041**. The second joins a, b, c, d as **0033**. endl adds a newline after each line; the source has no spaces in the output chain.

Memory rule to remember

Increment p: move to another element.
Increment *p: change the current element.
Prefix: use the updated value.
Postfix: use the old value.

Q32 | All address and value entries

Given: base address 0xACE0; 10 doubles; 8 bytes per double. All entries below are the four digits to write after the printed 0x. The first entry was already supplied in the paper.

Expression	Entry	Reason
doubleArray	ACE0	Decays to pointer to first element.
*doubleArray	0000	First element value is 0.
&doubleArray	ACE0	Address of the whole array.
&doubleArray[0]	ACE0	Address of element 0.
doubleArray[0]	0000	Element 0 has numeric value 0.
doubleArray+1	ACE8	Base + 1 x 8 bytes.
&doubleArray+1	AD30	Base + 10 x 8 = base + 0x50.
&doubleArray[1]	ACE8	Address of element 1.
doubleArray+9	AD28	Base + 9 x 8 = base + 0x48.
*doubleArray+9	0009	First value + 9 = 0 + 9.
*(doubleArray+9)	0022	Element 9 is decimal 34 = hex 22.
&doubleArray[9]	AD28	Address of element 9.
doubleArray[9]	0022	Element 9 is decimal 34.

Same address, different pointer types

```
doubleArray + 1   // double*: step by one double = 8 bytes
&doubleArray + 1  // double (*)[10]: step by 80 bytes
```

Both start at the same address, but the pointed-to types have different sizes. &doubleArray + 1 points just past the complete array; forming it is valid, but dereferencing it is not.

Do not confuse a value with its address

***doubleArray + 9** means **(*doubleArray) + 9**, giving numeric 9. ***(doubleArray + 9)** reads element 9, giving numeric 34. Only the pointer expressions represent addresses.

The old paper asks for the numeric element values in hexadecimal. It is not asking for the IEEE-754 binary representation of a double. Hence decimal 34 is written 0022, not as an encoded floating-point bit pattern.

Optional questions | Q10, Q17, Q23

Q10 | b - False

C++ also supports functional-style programming without third-party libraries. Built-in lambdas and function pointers allow callable values and functions that accept other functions. Therefore the claim that other paradigms require external libraries is false.

```
auto square = [](int x) { return x * x; };
int answer = square(5); // 25
```

Q17 | d - Green, 33

Split 0xFF202104 into bytes: alpha = FF, red = 20, green = 21, blue = 04. Shifting right by 8 moves the green byte to the lowest position; AND with 0xFF keeps just that byte. Hexadecimal 21 is decimal 2 x 16 + 1 = 33.

```
0xFF202104u >> 8        = 0x00FF2021
0x00FF2021 & 0xFFu      = 0x00000021
0x21                    = 33
```

Portability note: the paper uses signed int for a 32-bit color with the top bit set. The intended answer assumes the usual 32-bit representation/shift behavior. Using an unsigned 32-bit type makes this bit-extraction intent clearer and avoids older signed-conversion/shift portability issues.

Q23 | b, c - Stack and Queue

In the Stanford library documented for these collection classes, Stack and Queue do not support range-based for. Vector does, and Map supports iteration over keys. This question concerns that library's API, not a rule that every possible stack or queue class is forbidden to have iterators. [2-4]

Sources and verification

Question text: Exam Prep/Past Papers/2_CSC3002_Final_2021_Spring.pdf, original pp.2-8. Selection follows the earlier midterm packet in this chat. The workspace has since been reorganized; the original final was used to recheck the wording. No official solution sheet was supplied.

[1] C++ working draft, output streams: deleted copy constructor for basic_ostream.
https://eel.is/c++draft/output.streams

[2] Stanford C++ Library, Stack documentation.
web.stanford.edu/dept/cs_edu/resources/cslib_docs/Stack

[3] Stanford C++ Library, Queue documentation.
web.stanford.edu/dept/cs_edu/resources/cslib_docs/Queue

[4] Stanford C++ Library, Map documentation.
web.stanford.edu/dept/cs_edu/resources/cslib_docs/Map

References checked 26 October 2026. Library documentation corroborates Q23 but is not an archived official 2021 answer key.